Invoice Intake Pipeline

An LLM-based extraction step that turns unstructured invoice text into validated accounting records, auto-posts the clean ones, and sends the doubtful ones to a person for approval.

Who uses it and what it moves: Built for accounts-payable teams, it extracts vendor, line-item and total fields from invoices, auto-posts only the documents that pass schema, arithmetic and confidence checks, and routes the rest to a reviewer, targeting lower manual data-entry time per invoice and a lower exception rate.

Stage: Working prototype. The extraction, validation and review steps run end to end; the production wiring (webhook, AWS Lambda, PostgreSQL) is specified in section 6.

1. The problem

Most small and mid-size finance teams still key invoices in by hand. Every vendor uses a different layout, some invoices arrive as clean PDFs and others as blurry scans, and the arithmetic on the page is not always right. Typing is slow, and the real risk is not the typing itself but paying an amount that nobody checked.

A plain "send the PDF to an LLM" approach fixes the speed problem but creates a trust problem: the model will happily return a confident-looking number even when the page is unreadable. So the design goal here was narrower and more useful: let the model do the reading, but never let a record reach the ledger unless it can prove itself.

2. How it works

Each document moves through five stages. The first screenshot shows the landing view with the pipeline laid out across the top and a sample invoice loaded into the input panel.

Webhook	Extract	Validate	Route	Record
Invoice arrives from an inbox or upload	LLM returns fields plus a confidence score for each	Schema, arithmetic and confidence rules run	Auto-post, hold for review, or reject	Row written to Postgres; KPIs updated

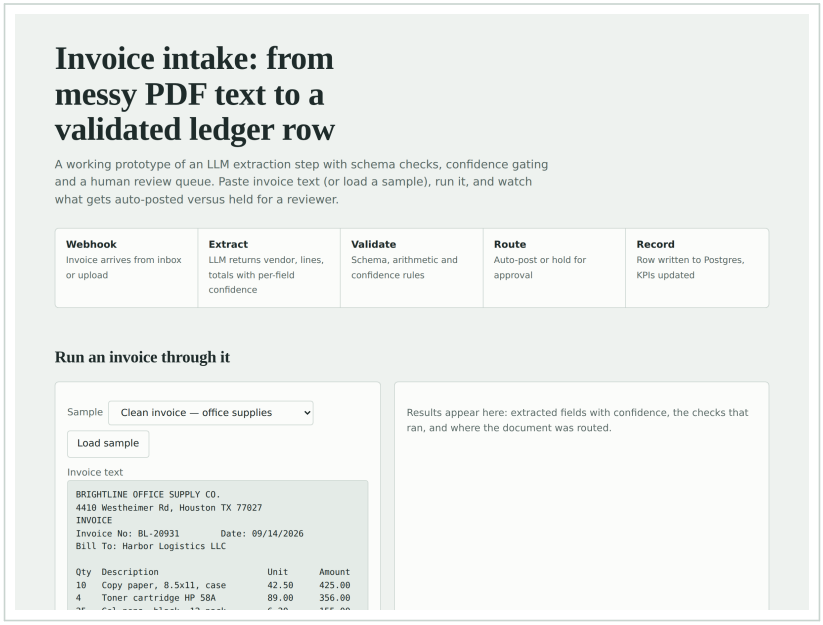


Figure 1. Landing view: the five pipeline stages, and the input panel with the "clean invoice" sample loaded.

3. Walkthrough: three invoices, three outcomes

I picked three test invoices on purpose so that each one exercises a different path through the rules: one that should sail through, one with a real arithmetic error, and one that is too damaged to trust. The goal of the test set is coverage of the routing logic, not a realistic mix of traffic.

Step 1. A clean invoice is auto-posted

An office-supply invoice with a normal layout. The model extracts the vendor, invoice number, date, three line items, subtotal, tax and total, and scores every field at 0.96 or higher. All five checks pass: the line items add up to the subtotal (936.00), subtotal plus 8.25% tax equals the total (1,013.22), and no field falls under the 0.85 confidence floor. Nobody needs to look at it, so it goes straight to the ledger.

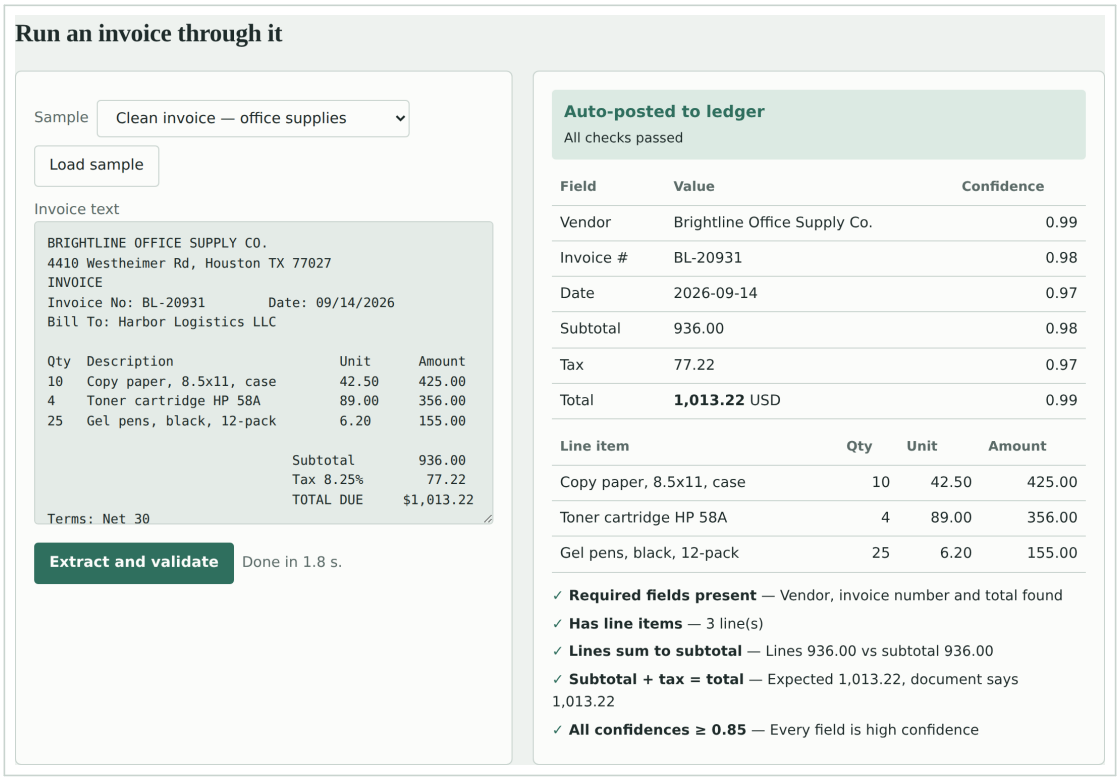


Figure 2. Clean invoice: every check passes and the document is auto-posted.

Step 2. A math error is caught and held

A freight invoice where the line items and subtotal agree (797.40) but the "amount due" is 815.40, with zero tax. The model reads every number correctly and with high confidence, which is exactly the case a confidence-only system would miss. The arithmetic check catches the 18.00 gap and holds the invoice for a person, instead of silently paying the higher figure.

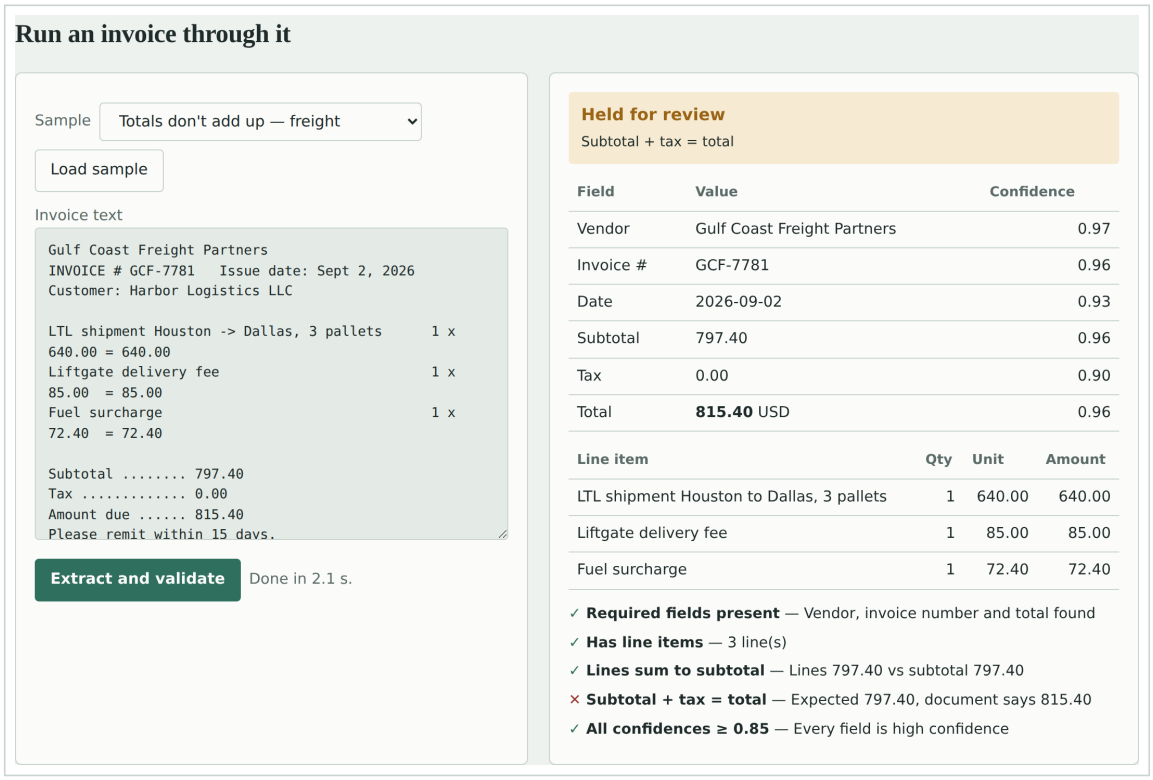


Figure 3. Totals don't reconcile: extraction is confident, but the arithmetic rule fails, so the invoice is held.

Step 3. An unreadable scan is rejected

A badly scanned IT-services invoice with OCR noise ("0" for "O", "1" for "l", question marks in place of digits). The prompt tells the model never to guess, so it returns null for the subtotal and total and low confidence on the invoice number. With no recoverable total there is nothing safe to post, so the document is rejected and would go back to the vendor for a clean copy.

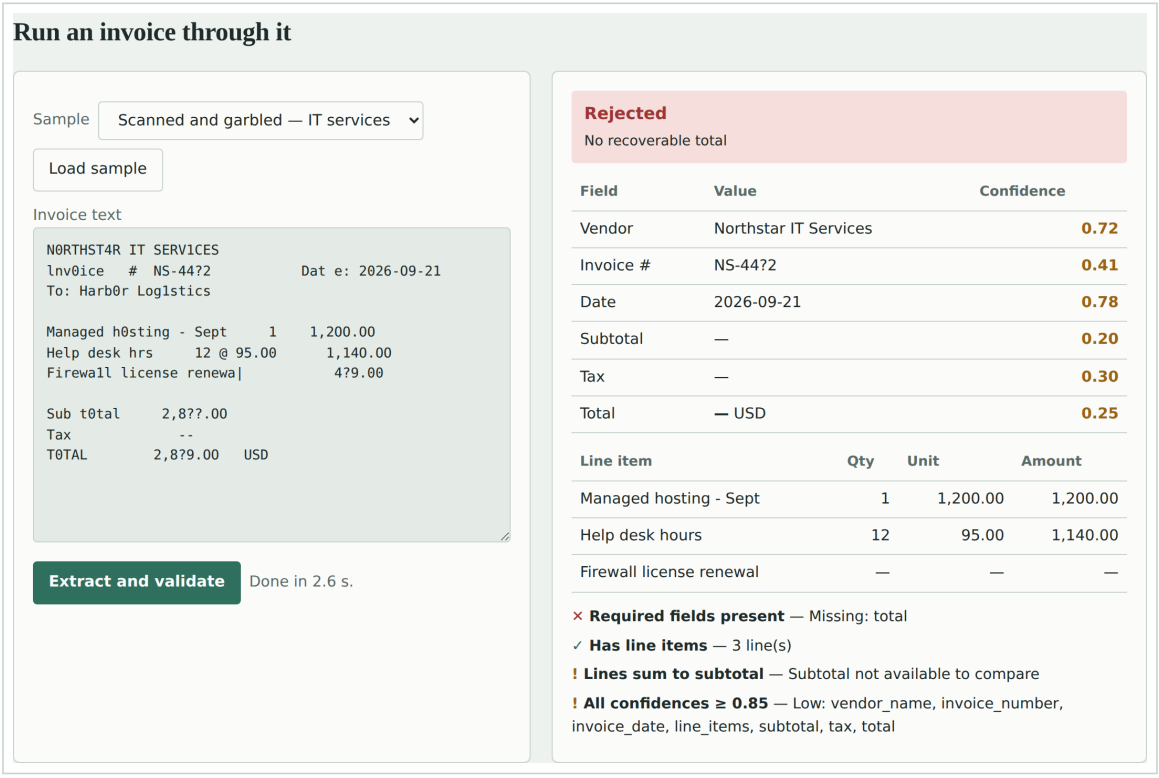


Figure 4. Garbled scan: the model declines to invent numbers, confidences drop, and the document is rejected.

Step 4. The reviewer works the queue

Everything processed lands in one table. Held items show Approve and Reject buttons with the reason they were held, so the reviewer only spends time on the documents that actually need judgment.

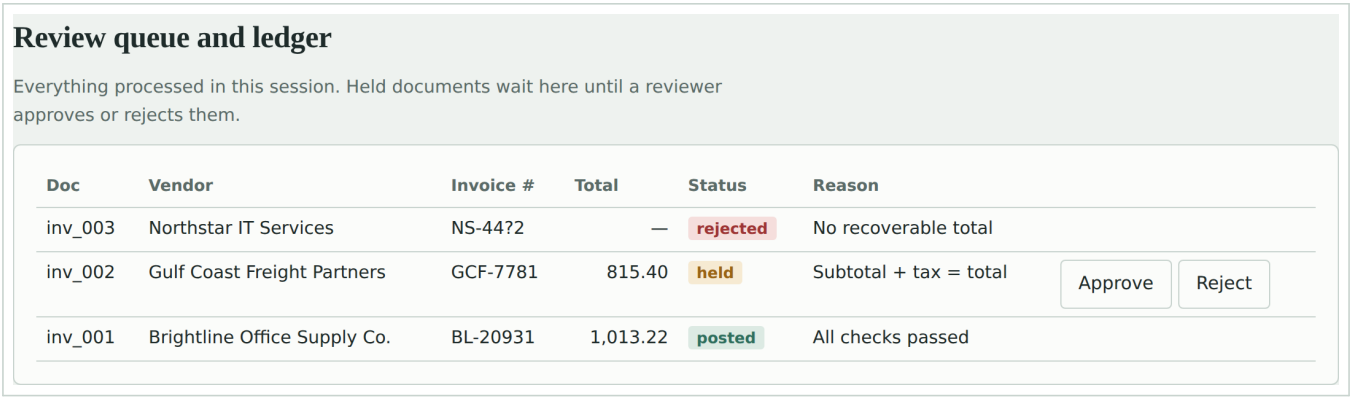


Figure 5. Review queue before action: the freight invoice waits for a decision.

Step 5. Approval updates the record

After the reviewer confirms the freight total with the vendor and approves it, the status changes to "approved" and the reason is logged as a reviewer decision, which keeps an audit trail of who overrode the rules and why.



Figure 6. After approval: one posted automatically, one approved by a person, one rejected.

Step 6. KPIs update as documents flow

The dashboard tracks documents processed, share auto-posted, exception rate, median cycle time and estimated model cost per document. With the three deliberately difficult test invoices above, the exception rate is 67%; that number reflects the test set, not expected production traffic, where clean invoices are the majority.

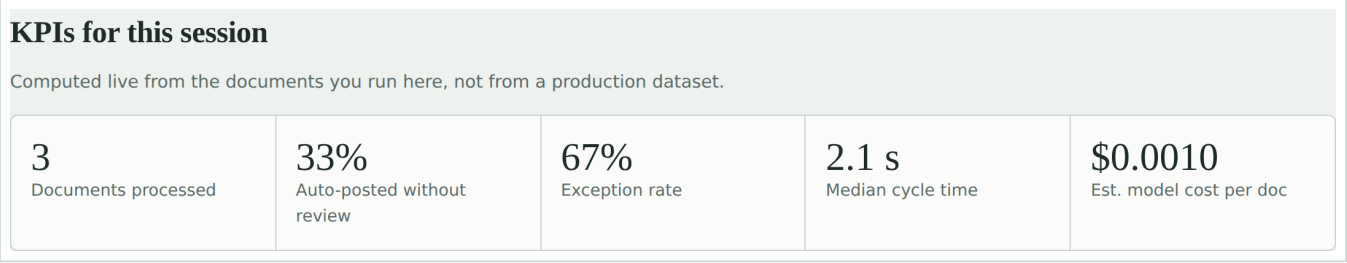


Figure 7. KPIs for the three-document test run. Cost is estimated from token counts.

4. Validation and routing rules

The rules are deliberately simple so a finance lead can read them and agree with them. A document is auto-posted only when all of the following are true:

Rule	What it catches	If it fails
Required fields present	Missing vendor, invoice number or total	Hold (reject if no total)
Has line items	Summary-only or truncated invoices	Hold
Lines sum to subtotal (±0.01)	Mis-read or missing line amounts	Hold
Subtotal + tax = total (±0.01)	Vendor math errors, hidden fees	Hold
Every field confidence ≥ 0.85	OCR noise and guessed values	Hold

Two design choices matter most. First, the arithmetic checks are independent of the model, so they catch errors even when the model is confident. Second, the prompt instructs the model to return null instead of guessing, which turns "silently wrong" into "visibly missing," and missing is easy to route.

5. API contract (reusable for purchase orders)

The extraction step sits behind one endpoint. Changing `doc_type` swaps the schema and rules, so the same handler can process purchase orders without new plumbing.

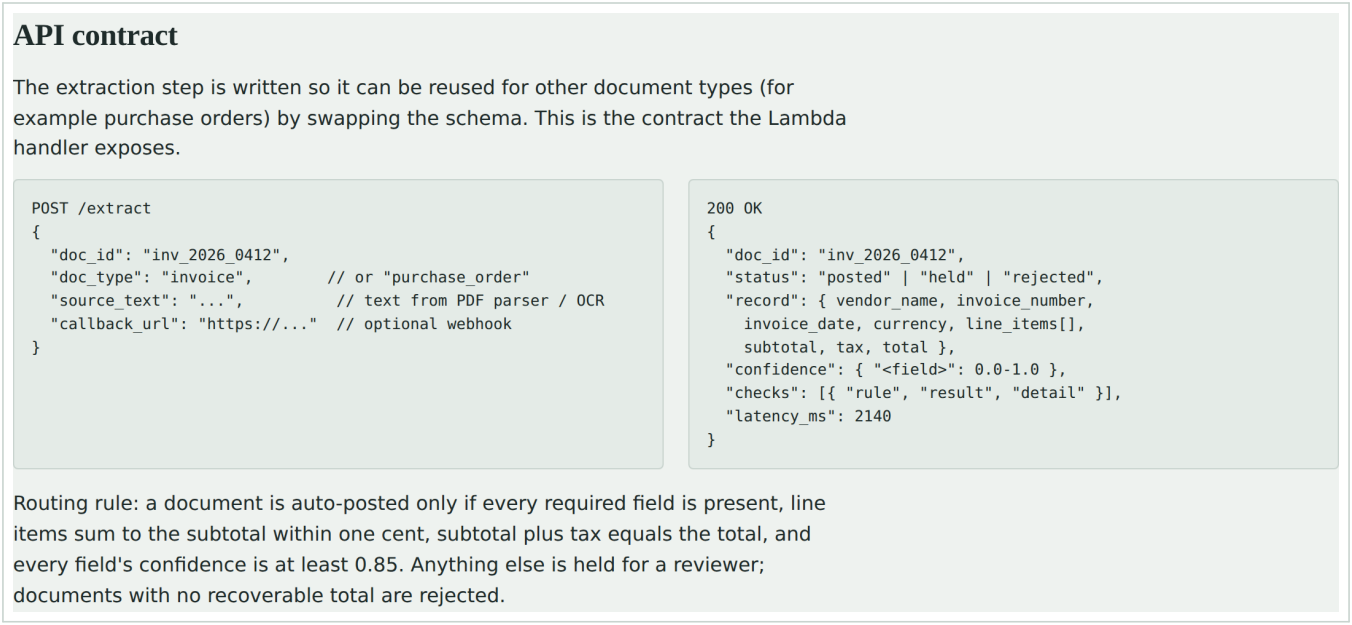


Figure 8. Request and response shape, and the routing rule, as documented on the demo page.

6. Production design

The demo runs in the browser. The production version is designed as follows: an inbox or upload webhook invokes an AWS Lambda function; the function converts the PDF to text (with OCR for scans), calls the LLM with the schema, runs the same checks, and writes to PostgreSQL. Posted

invoices go into the `invoices` and `invoice_lines` tables, held ones into a review table read by the approval screen, and a scheduled job rolls up cycle time, exception rate and cost per document for the dashboard.

How the production version is wired

In a deployed setup, an email or upload webhook calls an AWS Lambda function. The function parses the PDF to text, calls the LLM with the schema below, runs the same checks shown on this page, then inserts into PostgreSQL. Held rows go to a review table that an approval UI reads from, and a scheduled job rolls cycle time, exception rate and cost per document into the KPI dashboard.

```
CREATE TABLE invoices (
  doc_id          text PRIMARY KEY,
  vendor_name     text NOT NULL,
  invoice_number  text NOT NULL,
  invoice_date    date,
  currency        char(3),
  subtotal        numeric(12,2),
  tax             numeric(12,2),
  total           numeric(12,2) NOT NULL,
  status          text CHECK (status IN ('posted','held','rejected','approved')),
  min_confidence  numeric(3,2),
  received_at     timestamptz DEFAULT now(),
  posted_at       timestamptz
);
CREATE TABLE invoice_lines (
  doc_id text REFERENCES invoices, line_no int,
  description text, quantity numeric, unit_price numeric(12,2), amount numeric(12,2),
  PRIMARY KEY (doc_id, line_no)
);
```

Figure 9. Production wiring and the PostgreSQL schema.

7. Limitations and next steps

The current build accepts invoice text rather than raw PDF files, so PDF parsing and OCR are the next piece to add. The 0.85 confidence threshold was set by judgment and should be tuned against a labelled set of real invoices, measuring field-level accuracy and the share of documents sent to review. Duplicate-invoice detection (same vendor and invoice number) and matching against purchase orders are the two checks with the most value to add next.

Note on screenshots: figures were captured from a local run of the same page using recorded extraction responses for the three sample invoices, so the results are reproducible. In live use the model is called in real time, so confidence scores may differ slightly between runs.